Session 2: Preprocessing and Traditional Language Representation using scikit-learn tools.

Goal

The main goal of this session is to delve into the diverse techniques employed for preprocessing textual data in the field of Natural Language Processing. Additionally, it aims to familiarize students with traditional methods for representing textual data, particularly in the context of classification tasks. By comprehensively examining the advantages and limitations inherent in these learned representations, students can discern how to approach an NLP classification task.

Bibliography

- https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.TruncatedSVD.html.
- https://scikit-learn.org/stable/modules/feature_extraction.html#feature-extraction.
- Thomas K Landauer, Peter W. Foltz & Darrell Laham (1998). An Introduction to Latent Semantic Analysis. Discourse Processes, 25:2-3, 259-284, DOI: 10.1080/01638539809545028

PREPROCESSING

The first step in creating machine learning systems is to load the dataset (in this case, a text corpus). Then, preprocessing is one of the essential steps in many NLP tasks, such as topic classification, fake news detection, hate speech detection, sentiment analysis and many others. Usually, texts need to be transformed before use.

Some steps that can be useful in the preprocessing phase:

- Conversion of all letters to upper or lower case.
- Sentence segmentation and tokenization.
- Noise removal: unwanted characters, such as URLs, punctuation marks, special characters, etc.
- Removal non-content words (stopwords): The NLTK library can be used to obtain a list of stopwords for each language (https://www.nltk.org/).
- Stemming: Obtain the root of a word. For that, the PorterStemmer method provided by NLTK is widely used.
- Lemmatization: Obtain the standard form of words (lemma).

In English text processing, many robust tools exist, including:

1. NLTK
2. Spacy
3. Stanza
4. Freeling

Each tool is renowned for its effectiveness and versatility, and all of them can be used in Python.

ONE-HOT WORD ENCODING

One-hot word encoding is a technique used in NLP to represent words as binary vectors, where each word in a vocabulary is assigned a unique index. In this encoding scheme, the vector length is equal to the vocabulary size, and each word is represented by a vector with all elements being zero except for the element corresponding to its index, which is set to one. This encoding method is commonly used in tasks such as text classification and neural network-based language models.

In the provided corpus (a toy example) comprising three short documents:

```
D1: The cat sat on the mat.
D2: The dog chased the cat.
D3: The mat was soft and fluffy.
```

The one-hot encoding for the words is computed as follows:

Vocabulary (all words were transformed into lowercase):
`and, on, was, fluffy, sat, soft, mat, chased, cat, dog, the`

One-hot words encoding

```
D1:  The cat sat on the mat
the: [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
cat: [0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]
sat: [0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
on:  [0. 1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
the: [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
mat: [0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
```

```
D2 :    The dog chased the cat
the:    [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
dog:    [0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0.]
chased: [0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0.]
the:    [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
cat:    [0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0.]

D3:  The mat was soft and fluffy
the:    [0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
mat:    [0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
was:    [0. 0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
soft:   [0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
and:    [1. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
fluffy: [0. 0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
```

The approach to represent a document using one-hot word encoding is to aggregate the one-hot vectors for all words in the text. Applying this concept, the documents above can be represented as follows:

```
D1: [0. 1. 0. 0. 1. 0. 1. 0. 1. 0. 1.]
D2: [0. 0. 0. 0. 0. 0. 0. 1. 1. 1. 1.]
D3: [1. 0. 1. 1. 0. 1. 1. 0. 0. 0. 1.]
```

Note that in both D1 and D2, the word *'the'* appears twice. However, the aggregation strategy is applied in the document representation using logical OR. Based on this simple idea many text representation methods have been proposed.

TEXT REPRESENTATION

The written language (sequences of words) cannot be used directly by machine learning algorithms because most of them require numerical feature vectors with a fixed size. Raw texts with distinct lengths must be converted into fixed-sized numerical vectors (**vectors of features**). Some methods for transforming textual raw data into a numerical representation are:

BAG-OF-WORDS (BoW)

BoW is an easy and effective way to transform text into a numerical representation. In this representation, each text in the dataset is tokenized. Then, a vocabulary is obtained from all the tokens in the entire dataset. A vector is obtained for each text. Each axis represents the number of times that a token (in the vocabulary) appears in the text (its simplest version, a binary value, 1 if it occurs at least once, 0 otherwise).

Please Note:

The corpus is represented by a matrix with one row per text instance and one column per token in the vocabulary. It is a high-dimensional and sparse representation (each term in the vocabulary represents a feature). *The order of the words in the text is ignored.*

The **scikit-learn** library provides the **CountVectorizer** class. It implements both tokenization and frequency counting in a single class. Some parameters are:

- *max_df*: Ignore terms in the vocabulary with a frequency higher than the given threshold.
- *min_df*: Ignore terms in vocabulary with a frequency lower than the given threshold.
- *max_features*: Define the number of features to be considered in the representation.
- *vocabulary*: Define a vocabulary to be considered.
- *binary*: All non-zero counts are set to 1.

In the following examples, we utilize the corpus "*MyCorpus*" to illustrate various forms of text representation. For simplicity, we exclusively employ the corpus "*ShortCorpus*" to demonstrate character n-grams-based representation.

```
MyCorpus = ["Yo it shirt he gave New York was funny.",
"my husband is sick, homemade chicken soup loading husband.",
"Witch raises wind to break up enemy's lumber pound.",
"i got a new shirt at work at it is the WORST material ever",
"wave is so poor the girls can't even dress up for Halloween",
"The theory of paint indicates feelings of isolation in society.",
"Wish sometimes I had access to the ever elusive cock carousel..",
"off probation tomorrow ima be a free woman again, FUCK THE SYSTEM"
]
```

EXAMPLE 1: Binary Bag of Word

```
from sklearn.feature_extraction.text import
CountVectorizer

vectorizer = CountVectorizer(binary= True)
vectorizer.fit(MyCorpus)
X_bag_of_words = vectorizer.transform(MyCorpus)

#Print the computed vocabulary
print(vectorizer.vocabulary_)
print("Vocabulary size: ", len(vectorizer.vocabulary_))
for x in X_bag_of_words.toarray().astype(int):
  print(x)
```

```
{'yo': 65, 'it': 32, 'shirt': 45, 'he': 24, 'gave': 19, 'new': 37, 'york':
66, 'was': 57, 'funny': 18, 'my': 36, 'husband': 26, 'is': 30, 'sick': 46,
'homemade': 25, 'chicken': 7, 'soup': 50, 'loading': 33, 'witch': 61,
'raises': 44, 'wind': 59, 'to': 54, 'break': 4, 'up': 56, 'enemy': 11,
'lumber': 34, 'pound': 42, 'got': 21, 'at': 2, 'work': 63, 'the': 52,
'worst': 64, 'material': 35, 'ever': 13, 'wave': 58, 'so': 47, 'poor': 41,
'girls': 20, 'can': 5, 'even': 12, 'dress': 9, 'for': 15, 'halloween': 23,
'theory': 53, 'of': 38, 'paint': 40, 'indicates': 29, 'feelings': 14,
'isolation': 31, 'in': 28, 'society': 48, 'wish': 60, 'sometimes': 49,
'had': 22, 'access': 0, 'elusive': 10, 'cock': 8, 'carousel': 6, 'off':
39, 'probation': 43, 'tomorrow': 55, 'ima': 27, 'be': 3, 'free': 16,
'woman': 62, 'again': 1, 'fuck': 17, 'system': 51}

Vocabulary size:  67

[0000000000000000001100001000000010000100000001000000000001000000011]
[0000000100000000000000000011000100100100000000010001000000000000000]
[0000100000010000000000000000000000100000001010000000001010010100000]
[0010000000000100000001000000000101001010000000100000010000000000100]
[0000010001001001000010010000001000000000010000010000100010100000000]
[0000000000000010000000000000011010000001010000000100011000000000000]
[1000001010100100000000100000000000000000000000000100101000001000000]
[0101000000000000110000000001000000000000100010000000110010000001000]
```

As can be observed, the vocabulary has 67 unique terms. In this example, the stopwords cannot be removed. For each term, it showed the term and the index of the term in the vocabulary.

EXAMPLE 2: Bag of Word without stopwords

```
from sklearn.feature_extraction.text import
CountVectorizer

vectorizer = CountVectorizer(stop_words="english",
binary=False)
vectorizer.fit(MyCorpus)
X_bag_of_words = vectorizer.transform(MyCorpus)

#Print the computed vocabulary
print(vectorizer.vocabulary_)
print("Vocabulary size: ", len(vectorizer.vocabulary_))

#Showing the representation of each document in the
corpus
for doc in X_bag_of_words.toarray().astype(int):
    print(doc)
```

```
{'yo': 43, 'shirt': 30, 'gave': 12, 'new': 24, 'york': 44, 'funny': 11,
'husband': 17, 'sick': 31, 'homemade': 16, 'chicken': 3, 'soup': 33,
'loading': 21, 'witch': 39, 'raises': 29, 'wind': 37, 'break': 1, 'enemy':
7,
'lumber': 22, 'pound': 27, 'got': 14, 'work': 41, 'worst': 42, 'material':
23, 'wave': 36, 'poor': 26, 'girls': 13, 'dress': 5, 'halloween': 15,
'theory': 34, 'paint': 25, 'indicates': 19, 'feelings': 8, 'isolation': 20,
'society': 32, 'wish': 38, 'access': 0, 'elusive': 6, 'cock': 4,
'carousel': 2, 'probation': 28, 'tomorrow': 35, 'ima': 18, 'free': 9,
'woman': 40, 'fuck': 10}

Vocabulary size:  45

[0 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1]
[0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 2 0 0 0 1 0 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 1 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0]
[0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 1 0 0 0 0 0 0 1 0 1 0 0 0 0 0 0 0 0 0]
[1 0 1 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0]
```

Setting the *stop_words="english"* parameter reduces the vocabulary size to 45 unique terms. Then, the document

representations are computed based on this new vocabulary. Notice that, for the 2nd document, the word *'husband'* appears twice because the parameter *binary* is set to False.

N-GRAMS OF WORDS OR CHARACTERS (N-GRAMS)

The BoW representation cannot capture multi-word phrases because it does not consider word order. By considering sequences of adjacent words, n-gram-based representations can more effectively capture the contextual information and syntactic structure of language. Furthermore, BoW does not take into account possible misspellings in the words. A partial solution to these problems is a character n-grams representation.

N-grams-based Representation

Instead of building a simple vocabulary of unigrams (*n=1*), it can build a vocabulary of bigrams (*n=2*), where the occurrences of consecutive word pairs are counted.

Alternatively, we can consider a collection of n-grams of characters which is a robust representation for addressing some misspellings and informality in texts. For example, for the words in: *['words', 'wprds']* the character bigrams can be obtained:

words → _w, **wo, or**, rd, ds, s_
wprds → _w, **wp, pr**, rd, ds, s_

A BoW representation considers both words as different, while a bigrams of characters representation finds 4 out of 6 possible matches.

To obtain character n-grams, the **CountVectorizer** class can be used, but this time, the parameters **analyzer** and **ngram_range** need to be specified.

Parameters:

- *analyzer*: {'word', 'char', 'char_wb'} or callable, default='word'
- *ngram_range*: tuple (min_n, max_n), default=(1, 1)

```
ShortCorpus = ["my husband is sick", "call me sexist"]
```

EXAMPLE 3: Bigrams of characters

```
ngram_vectorizer = CountVectorizer(analyzer='char_wb',
ngram_range=(2, 2))
counts = ngram_vectorizer.fit_transform(ShortCorpus)
print(ngram_vectorizer.vocabulary_)
print("Vocabulary size: ",
len(ngram_vectorizer.vocabulary_))
for doc in counts.toarray().astype(int):
    print(doc)
```

Vocabulary

```
{' m': 3, 'my': 20, 'y ': 30, ' h': 1, 'hu': 13, 'us': 28, 'sb': 23, 'ba':
7, 'an': 6, 'nd': 21, 'd ': 10, ' i': 2, 'is': 15, 's ': 22, ' s': 4,
'si': 25, 'ic': 14, 'ck': 9, 'k ': 16, ' c': 0, 'ca': 8, 'al': 5, 'll':
18, 'l ': 17, 'me': 19, 'e ': 11, 'se': 24, 'ex': 12, 'xi': 29, 'st': 26,
't ': 27}

Vocabulary size:  31
```

Documents

```
[0 1 1 1 1 0 1 1 0 1 1 0 0 1 1 1 1 0 0 0 1 1 1 1 0 1 0 0 1 0 1]
[1 0 0 1 1 1 0 0 1 0 0 1 1 0 0 1 0 1 1 1 0 0 0 0 1 0 1 1 0 1 0]
```

Notice that using `analyzer='char_wb'`, the n-grams are built within the word boundaries, while `analyzer='char'` allows n-grams that span more than one word.

EXAMPLE 4: 5-grams of characters

```
ngram_vectorizer = CountVectorizer(analyzer='char_wb',
ngram_range=(5, 5))
counts=ngram_vectorizer.fit_transform(ShortCorpus)
print(ngram_vectorizer.vocabulary_)
print("Vocabulary size: ",
len(ngram_vectorizer.vocabulary_))
for doc in counts.toarray().astype(int):
   print(doc)
```

Vocabulary

```
{' my ': 4, ' husb': 1, 'husba': 10, 'usban': 14, 'sband': 11, 'band ': 7,
' is ': 2, ' sick': 6, 'sick ': 13, ' call': 0, 'call ': 8, ' me ': 3, '
sexi': 5, 'sexis': 12, 'exist': 9, 'xist ': 15}

Vocabulary size:  16
```

Documents:

```
[0 1 1 0 1 0 1 1 0 0 1 1 0 1 1 0]
[1 0 0 1 0 1 0 0 1 1 0 0 1 0 0 1]
```

EXAMPLE 5: Bigrams of words (removing stopwords)

```
ngram_vectorizer = CountVectorizer(analyzer='word',
ngram_range=(2,2),binary=False,
stop_words="english")
counts=ngram_vectorizer.fit_transform(MyCorpus)
print(ngram_vectorizer.vocabulary_)
print("Vocabulary size: ",
len(ngram_vectorizer.vocabulary_))
for doc in counts.toarray().astype(int):
   print(doc)
```

Vocabulary

```
{'yo shirt': 38, 'shirt gave': 25, 'gave new': 9, 'new york': 20, 'york
funny': 39, 'husband sick': 13, 'sick homemade': 27, 'homemade chicken':
12, 'chicken soup': 2, 'soup loading': 28, 'loading husband': 17, 'witch
raises': 34, 'raises wind': 24, 'wind break': 32, 'break enemy': 1, 'enemy
lumber': 6, 'lumber pound': 18, 'got new': 11, 'new shirt': 19, 'shirt
```

```
work': 26, 'work worst': 36, 'worst material': 37, 'wave poor': 31, 'poor
girls': 22, 'girls dress': 10, 'dress halloween': 4, 'theory paint': 29,
'paint indicates': 21, 'indicates feelings': 15, 'feelings isolation': 7,
'isolation society': 16, 'wish access': 33, 'access elusive': 0, 'elusive
cock': 5, 'cock carousel': 3, 'probation tomorrow': 23, 'tomorrow ima': 30,
'ima free': 14, 'free woman': 8, 'woman fuck': 35}

Vocabulary size:  40

Documents:
[0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 1 1]
[0 0 1 0 0 0 0 0 0 0 0 1 1 0 0 0 1 0 0 0 0 0 0 0 0 0 1 1 0 0 0 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 1 0 0 0 0 0]
[0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 1 1 0 0]
[0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0]
[0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 1 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0]
[1 0 0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0]
[0 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0 1 0 0 0 0]
```

TF-IDF (Term Frequency-Inverse Document Frequency)

It is a numerical measure (score) that quantifies the importance of the terms in a text given a corpus of documents. It has its origins in information retrieval systems. The idea with TF-IDF is to reduce the weight of terms proportionally to the number of documents in which they appear. In this way, a term's value increases proportionally to the number of times it appears in the text, and it is compensated by its frequency in the corpus. The scikit-learn library provides the classes TfidfTransformer and TfidfVectorizer for that purpose.

TfidfVectorizer converts a text corpus into a TF-IDF matrix, and **TfidfTransformer** transforms an existing frequency matrix into a TF-IDF representation.

Some parameters:

- ***norm****: {'l1', 'l2'} or None, default='l2'.* Each output row will have unit norm, either:

  - 'l2': Sum of squares of vector elements is 1. The cosine similarity between two vectors is their dot product when l2 norm has been applied.
  - 'l1': Sum of absolute values of vector elements is 1. See normalize.
  - None: No normalization.
- ***use_idf**: bool, default=True.* Enable inverse-document-frequency reweighting. If False, idf(t) = 1.

More at: manual page

EXAMPLE 6: Unigram of words with TF-IDF

```
from sklearn.feature_extraction.text import
TfidfVectorizer
from sklearn.feature_extraction.text import
TfidfVectorizer
tfidf_vectorizer =
TfidfVectorizer(stop_words="english", binary=False,
use_idf=True)
counts=tfidf_vectorizer.fit_transform(MyCorpus)
print(tfidf_vectorizer.vocabulary_)
print("Vocabulary size: ",
len(ngram_vectorizer.vocabulary_))
for doc in counts.toarray()
   print(np.round(doc, 2))
```

Vocabulary:

```
{'yo': 43, 'shirt': 30, 'gave': 12, 'new': 24, 'york': 44, 'funny': 11,
'husband': 17, 'sick': 31, 'homemade': 16, 'chicken': 3, 'soup': 33,
'loading': 21, 'witch': 39, 'raises': 29, 'wind': 37, 'break': 1, 'enemy':
7, 'lumber': 22, 'pound': 27, 'got': 14, 'work': 41, 'worst': 42,
'material': 23, 'wave': 36, 'poor': 26, 'girls': 13, 'dress': 5,
'halloween': 15, 'theory': 34, 'paint': 25, 'indicates': 19, 'feelings':
8, 'isolation': 20, 'society': 32, 'wish': 38, 'access': 0, 'elusive': 6,
```

```
'cock': 4, 'carousel': 2, 'probation': 28, 'tomorrow': 35, 'ima': 18,
'free': 9, 'woman': 40, 'fuck': 10}

Vocabulary size:  40
```

Documents:

```
[0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.43 0.43 0. 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.36 0.  0.  0. 0.
0.  0.36 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0. 0.  0.43 0.43]
[0.  0.  0.  0.41 0.  0.  0.  0.  0.  0.  0.  0.  0.  0. 0.  0.  0.41 0.41 0.  0.  0.  0.41 0.  0.  0.  0.  0.  0. 0.
0.  0.  0.41 0.  0.41 0.  0.  0.  0.  0.  0.  0.  0. 0.  0.  0. ]
[0.  0.38 0.  0.  0.  0.  0.  0.38 0.  0.  0.  0.  0.  0. 0.  0.  0.  0.  0.  0.  0.  0.  0.38 0.  0.  0.  0.  0.38
0.  0.38 0.  0.  0.  0.  0.  0.  0.  0.38 0.  0.38 0.  0. 0.  0.  0. ]
[0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0. 0.43 0.  0.  0.  0.  0.  0.  0.  0.43 0.36 0.  0.  0. 0.
0.  0.36 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.43 0.43 0.  0. ]
[0.  0.  0.  0.  0.  0.45 0.  0.  0.  0.  0.  0.  0.  0.45 0.  0.45 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.45 0.
0.  0.  0.  0.  0.  0.  0.  0.  0.45 0.  0.  0.  0.  0. 0.  0.  0. ]
[0.  0.  0.  0.  0.  0.  0.  0.  0.41 0.  0.  0.  0.  0. 0.  0.  0.  0.  0.  0.41 0.41 0.  0.  0.  0.  0.41 0.  0. 0.
0.  0.  0.  0.41 0.  0.41 0.  0.  0.  0.  0.  0.  0. 0.  0.  0. ]
[0.45 0.  0.45 0.  0.45 0.  0.45 0.  0.  0.  0.  0.  0.  0. 0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0.  0. 0.
0.  0.  0.  0.  0.  0.  0.  0.  0.  0.45 0.  0.  0. 0.  0.  0. ]
[0.  0.  0.  0.  0.  0.  0.  0.  0.  0.41 0.41 0.  0.  0. 0.  0.  0.  0.  0.41 0.  0.  0.  0.  0.  0.  0.  0. 0.41
0.  0.  0.  0.  0.  0.  0.41 0.  0.  0.  0.  0.41 0. 0.  0.  0. ]
```

Latent Semantic Analysis (LSA)

LSA is a technique used in NLP and information retrieval to analyze the relationships between a set of documents and the terms they contain. It is primarily employed for text classification, clustering, and information retrieval tasks.

LSA operates within the BoW vector space model, where documents and terms are represented as vectors in a high-dimensional space. Each dimension corresponds to a unique term, and each dimension's value represents the term's importance or frequency within the document.

This technique begins by constructing a term-document matrix, where rows represent terms and columns represent documents. The entries in the matrix typically correspond to some measure of the importance of each term in each document, such as term frequency-inverse document frequency values.

The **main idea behind LSA is to reduce the dimensionality of the term-document matrix while preserving the essential semantic relationships between terms and documents**. This is achieved through Singular Value Decomposition (SVD), a linear algebra technique.

The SVD technique decomposes the original term-document matrix into three separate matrices: $\mathbf{U}$, $\mathbf{\Sigma}$, and $\mathbf{V}^T$.

- $\mathbf{U}$ represents the relationship between terms in the reduced-dimensional space.
- $\mathbf{\Sigma}$ is a diagonal matrix containing singular values, indicating the importance of each dimension in capturing the variance in the data.

- $\mathbf{V^T}$ represents the relationship between documents in the reduced-dimensional space.

After SVD, LSA **retains only the top k singular values and their corresponding columns in matrices U and $\mathbf{V^T}$**, effectively reducing the dimensionality of the original term-document matrix. This reduces noise and captures the most significant semantic information.

Document and Term Representation

With the reduced-dimensional representation, documents and terms are now represented as vectors in the new “semantic space.” Documents are represented as vectors in the space spanned by the rows of $\mathbf{V^T}$, while terms are defined as vectors in the space spanned by the columns of the **U** matrix.

LSA is a powerful text representation technique that reduces the term-document matrix's dimensionality while preserving semantic relationships. It enables practical analysis and processing of text data in various NLP applications.

Although SVD is usually applied to the term-document matrix, if you want to use LSA to compute the similarity between documents, it is easier to apply SVD to the document-term matrix, as shown in the following example.

EXAMPLE 7: LSA document representation

```
from sklearn.feature_extraction.text import
CountVectorizer
from sklearn.feature_extraction.text import
TfidfTransformer
from sklearn.decomposition import TruncatedSVD
from sklearn.metrics.pairwise import cosine_similarity

#Document-Term Matrix
counter = CountVectorizer(stop_words="english",
binary=False)
dtMatrix = counter.fit_transform(MyCorpus).toarray()
print("After building the document-term",
dtMatrix.shape)

#Tf-idf Transformation
tfidf = TfidfTransformer()
tfidfMatrix = tfidf.fit_transform(dtMatrix).toarray()
print("After building the TFIDF matrix",
tfidfMatrix.shape)

#LSA Matrix
svd = TruncatedSVD(n_components = 5)
svdMatrix = svd.fit_transform(tfidfMatrix)
print("After Transforming the vectorial space with
LSA", svdMatrix.shape)
print(svdMatrix)
```

Output:

```
After building the document-term (8, 45)
After building the TFIDF matrix (8, 45)
After Transforming the vectorial space with LSA (8, 5)
[[ 7.93697112e-01 -1.57665586e-18  1.59931678e-17 -4.37048806e-18 -5.29860604e-17]
 [-3.06311420e-16  1.76403967e-01  4.00183513e-01  5.29812943e-01 6.16128333e-01]
 [ 5.24838991e-17  3.20438795e-01  1.45272336e-01 -1.44569892e-16 -1.73536345e-01]
 [ 7.93697112e-01  1.53703644e-17 -1.69190966e-17  4.77435084e-16 5.10173701e-17]
 [-1.78485818e-17  9.06900450e-01  1.04344702e-02  1.00803928e-16 -1.83597807e-01]
 [ 2.49589749e-16  1.41921072e-01  2.86696453e-02 -7.94719414e-01 3.11301492e-01]
 [ 5.07845459e-17 -1.20545612e-01  8.57693493e-01 -2.64906471e-01 -4.00450020e-02]
 [-5.91140840e-17  9.51806623e-02 -2.86670804e-01 -1.32453236e-01 6.76794384e-01]]
```

ACTIVITY

Implement the labeling, preprocessing, and text representations for the corpus "EXIST2026_training_subset.json". The corpus comprises the texts of memes labeled as SEXIST or NON-SEXIST. The JSON file has many fields, but only the following will be used:

Id_EXIST: It represents the id of the meme.
text: It represents the text of the meme.
labels_task2_1: It is a list of six labels, corresponding to the opinion of six different annotators. Each label can be "YES" or "NO", where "YES" means that the annotator considers the meme SEXIST, and "NO" means that the annotator considers the meme NON-SEXIST.

Labeling

Assign a unique label to each meme using a majority-voting scheme. The label of a meme is the label ("YES" or "NO") selected by at least four annotators. Note that in this subset all memes have a majority class, but this may not be the case in the full dataset.

Preprocessing

Do at least the following:

- Remove URLs.
- Convert all text to lowercase.

Text Representations

a) **Bag of Words (binary):** Represent the presence or absence of words in the text.
b) **Bag of Words (Term-Frequency):** Represent the text considering the term frequency of words without normalization.

c) **Bigrams of Words (Term-Frequency):** Represent the text considering the frequency of pairs of adjacent words without normalization.
d) **Trigram of Characters (Term-Frequency):** Represent the text considering sequences of three consecutive characters without normalization.
e) **TF-IDF:** Represent the text using the TF-IDF. In this case, the representation is based on words with 'l2' normalization.
f) **LSA:** Apply LSA to the previous TF-IDF representation, retaining the first 50 singular values.
g) **LSA:** Apply LSA to the previous TF-IDF representation, retaining the first 100 singular values.

You are required **to identify the most similar pair of samples for each representation**. It is important to emphasize that the most similar sample pairs must be determined **separately** for each class: **'SEXIST'** and **'NON-SEXIST'**. Specifically, you should find the most similar pair of texts among those memes classified as 'SEXIST' and another most similar pair among those memes classified as 'NON-SEXIST'. To measure the similarity between the text representations, you must use the **cosine_similarity** function provided by scikit-learn.

Tips

- Although the dataset is in JSON format, it may be useful to use a pandas DataFrame.
- If you do text preprocessing before vectorization, let the *preprocessor* parameter of the Vectorizer be equal to *None*.
- 'l2' normalization is the default normalization in *TfidfVectorizer.*

- **cosine_similarity** function expects an array-like argument of shape (n_samples_X, n_features) to compute the pairwise cosine similarity among all sample pairs.